

AT - ASSIGNMENT 1

Muhammad Hamza Bin Tariq - mb09640

GROUP 44

Problem 1 - Robot Navigation

Grid

4	S	.	.	#	.	.	.	S → starting position	Attempt to use the standard
3	#	#	.	#	.	#	.	.	→ empty tiles Cartesian Coordinate system
2	.	.	.	.	.	#	.	#	→ obstacle
1	.	#	#	#	.	.	.	G	→ goal
0	.	.	.	.	.	#	G		

0 1 2 3 4 5 6

formulation should clearly define

→ different: ① grid sizes

② starting position

③ goal positions

④ obstacle configurations

① Representation of a state $\rightarrow$ simply gives the current-cast.

② initial state $\rightarrow q$

① goal state $\rightarrow C$

④ possible actions $\rightarrow$ up, down, left, right

③ Successor Function ✓

⑥ Cost of each action ✓

⑦ Appropriate heuristic function for A^* → Euclidean

Class robot Navigation (Search Problem):

when creating an object, needs

① guide size

(2) starting position

② goal position

④ obstacle configuration (list of coordinates)

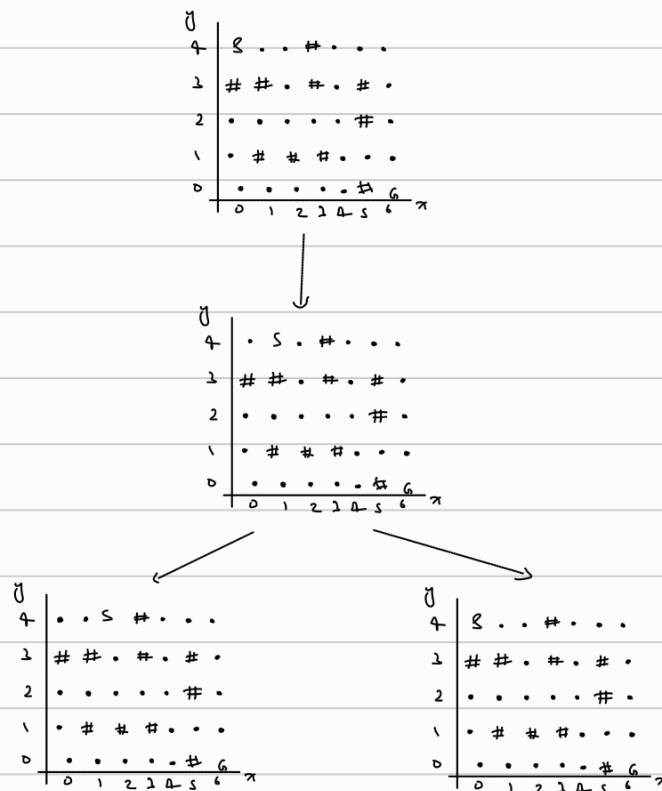
Allowed moves : up ($j-1$) $j = y$
 down ($j+1$) $i = x$
 left ($i-1$)
 right ($i+1$)

Objective $\rightarrow$ Find a path to the goal state

get Start State $\rightarrow$ just return start-pos

is Goal State $\rightarrow$ return state == self.goal-pos.

get Successors (self, state): $\rightarrow$ return possible states from the current state.



Possible Moves

$\Delta x, \Delta y$

y	x	0	1	2	3	4	5	6	x
4	S	.	.	.	.	.	.	.	
3	#	#	.	#	.	#	.	.	
2	.	.	.	.	.	.	#	.	
1	.	#	#	#	.	.	.	.	
0	.	.	.	.	.	.	.	#	G

1. Up: (0, 1)

2. Down: (0, -1) $\rightarrow$ Based on Cartesian Coordinates.

3. Right: (1, 0)

4. Left: (-1, 0)

$\rightarrow$ There can be a maximum of 4 successors.

$\rightarrow$ $0 \leq x \leq \text{self.width}$ and $0 \leq y \leq \text{self.height}$ and next-state not in self.obstacles.

$\rightarrow$ get Cost of Action (self, action)

$\rightarrow$ tracks current-state

$\hookrightarrow$ iteratively keeps checking till either an obstacle is in the way (return -1)

or action end (return total-cost).

For the heuristic function, we can use either Manhattan distance or Euclidean Distance

→ Using Euclidean Distance. $h(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

↳ rounded to 3 dp

General Dijkstra Algorithm.

→ Use a min queue

open = [] → priority — min queue → stores (state, distance)

initialize open queue with start_state (cost: 0)

initialize empty explored set.

while open queue isn't empty:

pop current_state with the minimum cost from open

if current_state is the goal:

return path

for each successor in get_successor(current_state):

if successor is in explored-set:

ignore and go to next

if successor is not in open:

push to open (state, path + new action, new cost)

elif successor is in open but new-cost < cost-in-open:

update successor in open with the cheaper new cost and new path

priority ↑
priority Queue will contain: ((state, complete-path, cost-for-node), cost-for-node)

Data	Priority
state, complete-path, cost-for-node	cost-for-node Dijkstra
state, complete-path, cost-for-node	cost-for-node + $h(n)$ → heuristic A*

* Some algorithm has A* as well, the priority value is the only thing that changes.

$$\begin{array}{ccc} \text{↳ } f(n) = h(n) + g(n) \\ \downarrow \qquad \qquad \downarrow \\ \text{heuristic} \qquad \text{cost-for-node} \end{array}$$

Problem 2 - Courier Delivery

Starting point $\rightarrow$ Redder

Connections $\rightarrow$ shows connections + weights (-1 for no connection)
 $\nearrow$ distance between two areas

heuristic $\rightarrow$ euclidean distance between every two areas

Track type $\rightarrow$ M (all vehicles)

S (all vehicles)

Dictionary Setup.

N (only bikes)

'Vertex': [('destination', distance, euclideanDistance, roadType), ...]

-1 (no connection)

$\rightarrow$ Find a route that only uses M or S, avoid N

return ① Path

② Total Cost

③ number of nodes expanded or visited.

csv files are like: , v1 v2 v3 ---

v1

v2

v3

⋮

getStartState() $\rightarrow$ returns just the name of the vertex

isGoalState() $\rightarrow$ returns state == self.goal-state

def getSuccessors() → return

(destination, destination, distance)

def getHeuristic() → return euclidean distance from current-state to goal state.

↳ update the graph to remove heuristic.

→ New graph structure: Origin: (Destination, Distance, Track Type)

→ Make a new dict to store heuristics



structure: (Origin, destination): heuristic

Make getHeuristic() for easier to implement as

the main graph only stores edges where distance != 0, distance != -1, tracktype != -1

→ another change to the main graph

↳ distance is now cost

↳ cost = distance * weight

track type	weight
M	1
S	1.2
N	1.8 (heavily de-incentivise the narrow routes

as they also have to change to bike
carriers)

A* with stopover

```
current_state = problem.getStartState()
```

```
hub = problem.getStartState()
```

```
unvisited_stopovers = stopovers.copy()
```

We will visit the stopovers based on their heuristic values (Greedy Best First)

```
complete_action = []
```

```
total_cost = 0
```

```
visit_order = []
```

while unvisited_stopovers:

```
    closest_stopover = None
```

```
    min_heuristic_distance = float('inf')
```

```
    for stopover in unvisited:
```

```
        h_cost = problem.heuristicDict[(current_location, stopover)] → Access the heuristic dictionary
```

```
        if h_cost < min_heuristic_distance:
```

```
            min_heuristic_distance = h_cost
```

```
            closest_stopover = stopover
```

The closest stopover becomes our next destination

```
next_goal = closest_stopover
```

```
unvisited_stopovers.remove(next_goal)
```

```
visit_order.append(next_goal)
```

Run A*

```
problem.start_state = current_location
```

```
problem.goal_state = next_goal
```

```
path, nodes, cost = search.alterSearch(problem)
```

store results

complete_action += path

total_cost += cost

go to the next destination for the next iteration

current_location = next_goal

back to hub

problem.start_state = current_location

problem.goal_state = hub

final_path, final_nodes, final_cost = search_after_search(problem)

complete_action += final_path

total_cost += final_cost

return complete_action, total_cost, visit_order

OPTIMIZATION

- Pick random start (x_0) inside the function bounds
- Track best point so far
- Outer loop while temp > 0
 - Inner loop run K times
 - random neighbour for each iteration (or modify function)
 - delta
 - if better $\rightarrow$ always accept $\rightarrow$ update best if needed
 - if worse $\rightarrow$ accept with prob $e^{-\frac{\text{delta}}{\text{temp}}}$
 - After K runs, $t = t - 0.1$
- return (x_{best})

Params

$T = 1.0 \rightarrow$ higher T means accepting worse moves more often

$N = 0.5 \rightarrow$ larger $N \rightarrow$ larger jumps

$K = 100 \rightarrow$ larger $K \rightarrow$ more searches at each T .